

# KD-BERT: A Lightweight Knowledge Distillation Bidirectional Encoder Representations from Transformers for IoT Network Intrusion Detection

Zhong Cao , Xiaohua Liu, Zheng Zhou , Lei Ding, and Wenli Shang 

**Abstract**—The Internet of Things (IoT) network intrusion detection (NID) has emerged as a powerful technique for effectively detecting attacks and protecting internet safety across a wide range of applications. However, existing NID models often struggle to achieve good detection performance when deployed in resource-restricted environments. To achieve an optimal balance between detection performance and computational cost, we propose a lightweight knowledge distillation bidirectional encoder representations from Transformers (KD-BERT) framework for efficient IoT intrusion detection. Specifically, we first develop a header-payload pairs tokenization method to preprocess the raw data. Then, we distill rich knowledge from a large pre-trained BERT<sub>teacher</sub> model into a smaller BERT<sub>student</sub> model with fewer training parameters. By leveraging KD-BERT, the BERT<sub>teacher</sub> can be compressed into the BERT<sub>student</sub>, reducing the number of parameters from 135 to 9 M, while maintaining high detection accuracy. To effectively obtain the essential features, we design a data transformation strategy and fine-tuned the BERT<sub>student</sub> model on several labeled IoT NID tasks. Comprehensive experimental results on three public datasets (telemetry datasets, operating systems datasets and network traffic dataset of IoT (TON-IoT), Edge-IIoTset, and IoMT2024) have demonstrated superior performance compared to state-of-the-art methods.

**Index Terms**—Intrusion detection, bidirectional encoder representations from Transformer (BERT, Knowledge distillation).

## I. INTRODUCTION

WITH the proliferation of the Internet of Things (IoT), its integration into daily life across industries, such as healthcare, pharmacy, food and beverage, agriculture, electronics, transportation, and retail, has become ubiquitous, harnessing its immense potential for practical solutions [1], [2], [3], [4], [5]. However, this widespread adoption has also attracted the attention of malicious actors, leading to a surge in IoT device intrusions and attacks [2]. These attacks have manifested in various forms, posing significant risks to users and society. For instance, hackers can infiltrate wireless webcams to violate privacy, compromise medical equipment endangering patients' lives, or access sensitive data from children's smartwatches and vehicle control systems, manipulating their functionality with dire consequences. A recent case underscores this threat, as a survey revealed that smart home systems experienced over 12 000 illicit attacks within a week, highlighting the urgent need for robust security measures to safeguard IoT devices and their users. Therefore, it is imperative to develop a lightweight IoT network intrusion detection systems (NIDS) to effectively detect and mitigate the emerging cyber threats.

IoT systems present distinct challenges for intrusion detection when compared to traditional networks, primarily due to their inherent constraints in computational power, memory capacity, battery life, and network bandwidth. Furthermore, the integration of a diverse range of heterogeneous protocols and technologies within the same system exacerbates these challenges. Previous intrusion detection models primarily relied on datasets with manually selected features. However, network traffic feature sets provided by various edge devices often exhibit inconsistency, and there is no standardized network traffic feature set in this field. As a result, feature engineering-based intrusion detection approaches face significant challenges in manually extracting features from the diverse range of protocols and technologies. In contrast, end-to-end approaches offer a promising direction for intrusion detection in IoT environments, as they have the

Received 11 March 2025; revised 6 May 2025; accepted 11 June 2025. Date of publication 22 July 2025; date of current version 17 October 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB2012300, in part by the National Natural Science Foundation of China under Grant 62173101, in part by the Basic and Applied Basic Research Funding of Guangdong Province under Grant 2022A1515011558 and Grant 2022A1515010865, in part by the Guangzhou Science and Technology Funding under Grant 202201020217, in part by the Guangzhou Basic and Applied Basic Research Topics under Grant 2025A04J3377, and in part by the Key Laboratory of On-Chip Communication and Sensor Chip of Guangdong Higher Education Institutes under Grant 2023KSYS002. Paper no. TII-25-1470. (Zhong Cao and Xiaohua Liu contributed equally to this work.) (Corresponding authors: Zheng Zhou; Wenli Shang.)

Zhong Cao, Xiaohua Liu, Zheng Zhou, and Wenli Shang are with the School of Electronics and Communication Engineering, Guangzhou University, Guangzhou 510006, China (e-mail: zhongc@gzhu.edu.cn; 32119500035@e.gzhu.edu.cn; zhouzheng@gzhu.edu.cn; shangwl@gzhu.edu.cn).

Lei Ding is with the School of Cyber Security, Guangzhou University, Guangzhou 510006, China (e-mail: 1112206007@e.gzhu.edu.cn).

Digital Object Identifier 10.1109/TII.2025.3582375

capability to discover globally optimal solutions without relying on manually selected features [6]. This makes them more adept at handling heterogeneous systems and addressing the complexities of IoT environments. However, these methods often require significant computational resources, making them impractical for direct application to IoT environments [1], [7], [8].

In this work, we introduce a lightweight knowledge distillation bidirectional encoder representations from Transformers (KD-BERT) framework specifically tailored for IoT environments. Our approach initially learns generalized representations from an extensive corpus of unlabeled IoT traffic. Subsequently, we compress this pretrained model into a lightweight student model, maintaining high performance through the application of knowledge distillation techniques. This student model is then fine-tuned for specific downstream tasks. Notably, we construct our pretraining dataset directly from raw packet-level traffic, enabling the model to discern inherent patterns within IoT networks. During pretraining, we introduce a same-origin packet prediction module, which effectively captures the intricate relationships between headers and payloads. Following pretraining, we utilize a BERT model consisting of 12 Transformer layers as a teacher to guide the training of a more efficient student BERT model containing only two Transformer layers. To obtain a compact student model, we strategically reduce the number of self-attention (SA) modules. Finally, KD-BERT fine-tunes its parameters using a limited amount of task-specific labeled data, ensuring optimal performance for the intended application. The main contributions of this article can be summarized as follows.

- 1) We propose a KD-BERT framework for IoT network intrusion detection, avoiding manual feature extraction and significantly reducing model computational complexity.
- 2) We implement a header-payload pairs tokenization strategy for preprocessing raw data, and devise a knowledge distillation module to capture essential representations from pivotal network layers. In addition, we incorporate a data transformation module utilizing a sliding window approach to fine-tune the KD-BERT model, further obtaining representative features and enhancing its performance.
- 3) We conduct comprehensive comparison experiments with state-of-the-art methods on real network security datasets (telemetry datasets, operating systems datasets and network traffic dataset of IoT (TON-IoT), Edge-IIoTset, and IoMT2024) to demonstrate the superiority and high-efficiency of KD-BERT.

The rest of this article is organized as follows. Section II introduces the related works of this article. In Section III, we propose the KD-BERT framework for IoT network intrusion detection. In Section IV, we evaluate the performance of the proposed approach and compare it with the state-of-the-art methods, demonstrating the superiority and effectiveness of our approach. Finally, we conclude this article in Section V.

## II. RELATED WORK

### A. Knowledge Distillation

Knowledge distillation [9] is a highly effective technique for compressing deep learning models through a transfer learning

process. In this approach, a larger, more complex teacher model guides the training of a smaller, more compact student model, enabling the student to mimic the functionality of the teacher. This technique has garnered significant research attention due to its remarkable ability to substantially reduce model sizes while maintaining high levels of accuracy. Wang et al. [10] introduced the concept of knowledge distillation into a triple convolutional neural network (TCNN) architecture, termed KD-TCNN, with the aim of deriving optimal features while utilizing a compressed student network. This approach effectively reduces resource consumption and model parameters, enabling real-time detection capabilities. Meanwhile, Wang et al. [11] developed BERT of Theseus Transformer and PoolFormer, which utilizes knowledge distillation in conjunction with a Siamese network dimension reducer and a large teacher model. This method successfully compresses an efficient detection model for IoT applications, demonstrating the suitability of knowledge distillation for IoT NIDS. By enabling model compression, knowledge distillation facilitates the practical deployment of NIDS in IoT environments.

### B. Transformer

The Transformer model [12] has achieved remarkable performance on a wide range of natural language processing tasks by incorporating the attention mechanism, distinguishing it from traditional deep learning approaches that rely on recurrent or convolutional layers. Subsequently, BERT [13] further advanced this field by leveraging a bidirectional Transformer architecture that enables the sharing of attention and parameters, resulting in outstanding multitask fine-tuning capabilities. These innovative models have found widespread application in IoT NIDS. Ferrag et al. [14] introduced a novel approach for cyber threat hunting in 6G IoT NIDS, combining the power of generative adversarial networks with a Transformer-based model. Their work focused on threat mining using a new dataset, demonstrating the potential of this methodology. In Ferrag et al. [15] they presented SecurityLLM, which builds upon BERT from the ground up, integrating it with a large language model called FalconLLM for enhanced threat analysis. Furthermore, they introduced PPFL, a structured and private format for representing network data, addressing the challenges posed by nontextual data for language models. Basset et al. [16] proposed federated deep learning-based intrusion detection system (FED-IDS), a framework that utilizes federated deep learning for collaborative detection of cyberattacks. Their approach incorporates an improved Transformer model to extract features for edge node detection, while ensuring data privacy is preserved.

Furthermore, several studies have delved into the application of end-to-end models grounded in Transformer architectures and their derivatives, directly processing raw traffic data. He et al. [17] introduced a payload encoding representation from Transformer, leveraging dynamic word embedding techniques to autonomously discern the salient features within traffic payloads. Similarly, Lin et al. [18] presented encrypted traffic bidirectional encoder representation from Transformer (ET-BERT), a groundbreaking traffic representation model that introduces a tailored traffic representation method alongside two traffic-oriented

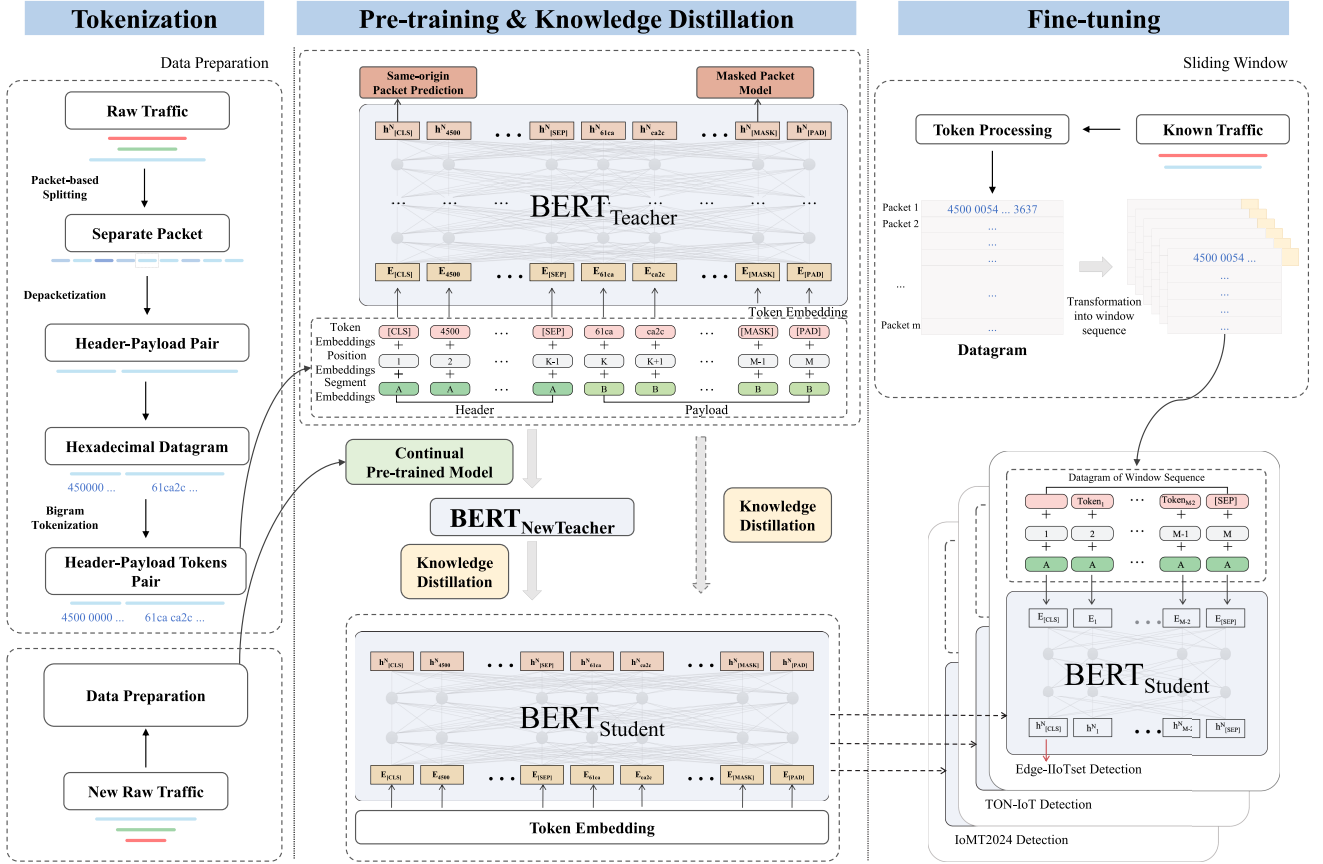


Fig. 1. Overall framework of KD-BERT for IoT network intrusion detection

self-supervised pretraining tasks: Masked BURST Model and Same-origin BURST Prediction.

Previous research has demonstrated significant advancements in applying knowledge distillation techniques and Transformer models for IoT network intrusion detection and traffic analysis. However, most methods rely on preprocessed datasets with manually crafted features, which limits their generalization capabilities in diverse IoT environments. Due to the constrained computational resources prevalent in IoT environments, end-to-end methodologies cannot be seamlessly integrated into IoT NIDS without appropriate adaptations. Therefore, it is necessary to develop a lightweight, efficient, and heterogeneous data-capable intrusion detection model within the context of IoT security.

### III. METHODOLOGY

#### A. Overall Model Architecture

We introduce KD-BERT, as shown in Fig. 1, an innovative framework that harnesses knowledge distillation to enhance BERT, empowering efficient and real-time network intrusion detection tailored for Industry 4.0 environments amidst computational constraints and IoT device heterogeneity.

IoT raw traffic data typically comprise unencrypted datagrams, rich in information but inherently challenging to interpret compared to curated IoT network data. To mitigate the model's complexity while leveraging the raw IoT network traffic for

intrusion detection and efficiently capturing the essential characteristics of IoT traffic, we propose the KD-BERT model as illustrated in Fig. 1.

1) The header-payload pairs tokenization module is firstly designed to preprocess the raw traffic data and elevate the KD-BERT model's proficiency in comprehending heterogeneous IoT data features. 2) The masked packet modeling and same-origin packet prediction pretraining tasks then fortify the KD-BERT model's understanding of the intricate relationships between IoT packet datagrams. 3) Furthermore, the knowledge distillation module is integrated into the BERT model to address the constraints of limited resources in IoT environments, yielding a resource-efficient KD-BERT student model. 4) Finally, we introduce a refined fine-tuning strategy utilizing window sequences, enabling the KD-BERT model to better acclimate to the complexities of IoT environments.

#### B. Header-Payload Pairs Tokenization

Given the inherent challenges of IoT traffic, which lack clear semantic units and human-comprehensible content compared to natural language and imagery, it significantly diverges from these domains. Furthermore, IoT systems often integrate diverse and heterogeneous protocols and technologies within the same ecosystem [7]. To address these complexities and enable our model to broadly learn the generalized representation of IoT

traffic during the pretraining phase, we devise an enhanced traffic representation framework.

To refine our approach, we initially segment the traffic into discrete packets, each of which is further divided into a header and payload section. For traffic that solely comprises headers or adheres to protocols, such as Bluetooth, we designate the entire packet as the header and augment the payload section with a dedicated [PAD] token to maintain a consistent structure. In cases where the payload exceeds 80 bytes, we truncate it at this threshold to ensure efficient processing. To mitigate potential interference and enhance anonymity, we implement a series of sanitization measures. Specifically, we replace the original IP addresses with randomly generated ones, reset all port numbers to zero, and strip the Ethernet headers from the packets. These steps not only reduce unnecessary information but also contribute to preserving the privacy of the data sources.

We tokenize raw IoT traffic datagrams into a sequence of units, each comprising two adjacent hexadecimal bytes. These tokens are then represented using byte-pair Encoding, with a range of 0 to 65535, supporting a maximum vocabulary size of 65 536. To enhance pretraining tasks, we incorporate special tokens: [CLS] as the initial token in each sequence, [SEP] to delineate headers from payloads, [PAD] for padding to meet fixed minimum length requirements, and [MASK] for context-based learning during pretraining.

Finally, a comprehensive tag representation is constructed utilizing three distinct word encodings: token embedding, position embedding, and segment embedding, to serve as the input representation for pretraining, which are depicted in Fig. 1. Following  $N$  iterations of Transformer encoding, where  $N$  varies between the student and teacher models, the refined final token embedding is derived.

### C. KD-BERT

*Masked packet mode:* During this phase, a random masking process is applied to the input sequence tokens, with a probability of 15%. Of the selected tokens, 80% are replaced with the [MASK] token, 10% are substituted with a random token, and the remaining 10% are left unchanged. This strategy aims to capture the intricate relationships within binary datagrams while fostering the model's ability to comprehend and generalize from partially obscured data. The objective function is defined as follows:

$$\mathcal{L}_{\text{MPM}} = - \sum_{i=1}^k \log (P(\text{MASK}_i = \text{token}_i | \bar{X}; \theta)) \quad (1)$$

where the probability  $P$  is modeled by the Transformer encoder equipped with trainable parameters  $\theta$ .  $\bar{X}$  represents the encoded representation of the masked input  $X$ , and  $\text{MASK}_i$  indicates the masking token at position  $i$  in the sequence.

*Same-origin packet prediction:* In the diverse landscape of IoT protocols, understanding the overarching traffic patterns and intrapacket connections is paramount. To achieve this, we emphasize the intimate interplay between the header and payload components within individual packets. To this end, we adopt the same-origin packet prediction approach, leveraging a binary

classifier that meticulously examines the relationship between header and payload. This classifier is tasked with distinguishing whether the header and payload belong to the same packet or originate from separate ones. To ensure robustness and generalizability, we introduce a balanced sampling strategy where, for each packet formation, there is an equal chance (50%) that the payload is sourced from the same packet as the header or from a distinct packet. The label  $y$  assigned to each input sequence succinctly indicates this origin congruence, facilitating precise model training and evaluation. The loss function is defined as follows:

$$\mathcal{L}_{\text{SPP}} = - \sum_{j=1}^n \log (P(y_j) | B_j; \theta) \quad (2)$$

where  $\theta$  are the trainable parameters and  $B_j$  represents the  $j$ th header–payload pair in the training data.  $y_j$  represents the definitive label that indicates whether the two constituent parts of the  $j$ th sentence pair are indeed corresponding to each other.  $P$  designates the model's predicted probability for the label of the  $j$ th pair to be  $y_j$ .

*Continue-pretraining:* Previous research has demonstrated that starting from scratch when training BERT on domain-specific corpora, such as the biomedical domain, and utilizing a custom vocabulary tailored to that space can significantly improve model's performance for the tasks within that domain [19]. The diversity of IoT systems and protocols results in vastly different network traffic patterns, which can be further altered by system upgrades. Consequently, a single dataset may not adequately represent the full spectrum of traffic types, necessitating a more nuanced approach to data collection and analysis. To address these challenges, we incorporate continue-pretraining into our methodology. We refine the teacher model, initially pre-trained on a foundational dataset, with a specialized new dataset. This approach facilitates adaptability to new environments and evolving systems, ensuring representation quality is preserved even with limited data, offering an advantage over training from scratch with small datasets.

*BERT distillation:* As shown in Fig. 2, we present a refined BERT distillation framework tailored for efficiently distilling knowledge into student models of any size. Given a teacher model with  $N$  Transformer layers, we obtain a student model by adaptively learning  $M$  layers from the teacher model to facilitate targeted knowledge distillation ( $N > M$ ). There is a function  $n = g(m)$ , which is defined as the mapping between the student layers and the teacher layers. The  $g(m)$ th layer of the teacher model is learned by the  $m$ th layer of the student model. We use  $g(m) = 6 \times m$  as the layer mapping function, so student model learns from every six layers of teacher model. Formally, the student model optimizes the following objective function to minimize the distillation loss:

$$\mathcal{L}_{\text{model}} = \sum_{m=1}^M \lambda_m \mathcal{L}_{\text{Trans}} \quad (3)$$

where  $\mathcal{L}_{\text{Trans}}$  denotes the loss function of a Transformer layer.  $\lambda_m$  denotes the hyperparameter that represent the importance of distillation at the  $m$ th layer of the Transformer.



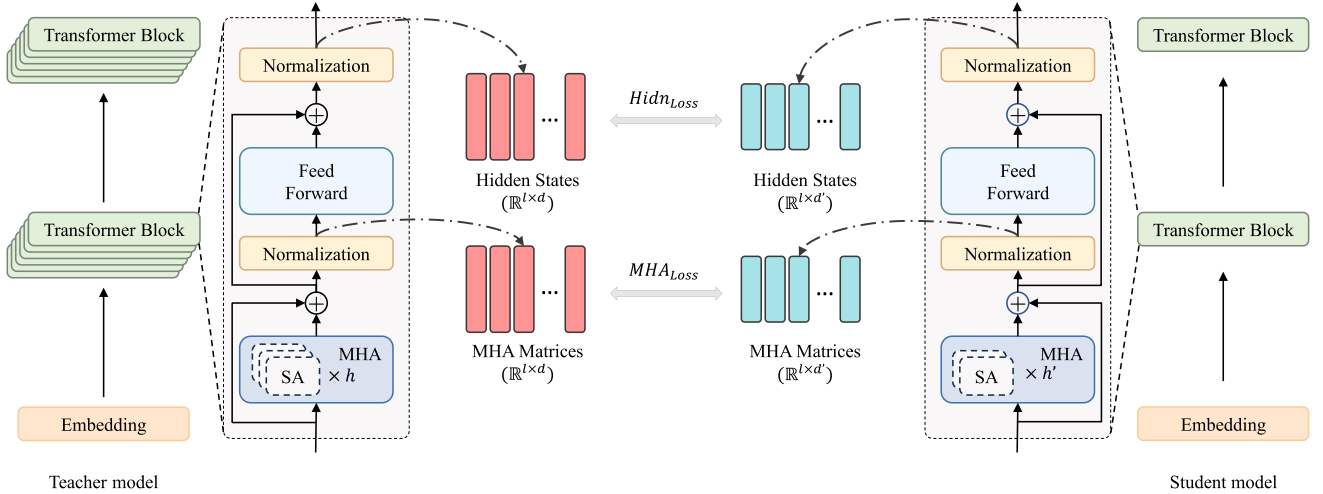


Fig. 2. Detailed structure of knowledge distillation on Transformer networks

The multihead attention (MHA) and hidden states-based distillation are included in the proposed Transformer-layer distillation. The complexity of the MHA mechanism is  $\mathcal{O}(HN^2D)$ , where  $N$  represents the length of the input sequence,  $D$  denotes the hidden-layer dimension, and  $H$  represents the number of heads. To further reduce the complexity of BERT, we simultaneously reduce the number of Transformer layers and the number of SA in MHA module. The student model adjusts its MHA parameters to closely mimic the MHA parameters of the teacher model, which are defined as follows:

$$\mathcal{L}_{\text{MHA}} = \text{MSE}(M^S W_M, M^T) \quad (4)$$

where the matrices  $M^S \in \mathbb{R}^{l \times d'}$  and  $M^T \in \mathbb{R}^{l \times d}$  refer to the MHA parameters of student and teacher networks, respectively. The scalar values  $d$  and  $d'$  denote the hidden size of teacher and student models ( $d' < d$ ), respectively. In our work, we utilize the ratio  $d = 3d'$  to derive a more concise student network.  $\text{mse}(\cdot)$  means the mean squared error function. The matrix  $W_M \in \mathbb{R}^{d' \times d}$  serves as a learnable linear transformation, enabling the student network's multihead attention weights to align with the space of teacher network.

Apart from the MHA based distillation, we also distill knowledge from the output of Transformer layers using the following objective function:

$$\mathcal{L}_{\text{Hidn}} = \text{MSE}(H^S W_H, H^T) \quad (5)$$

where the matrices  $H^S \in \mathbb{R}^{l \times d'}$  and  $H^T \in \mathbb{R}^{l \times d}$  refer to the hidden parameters of student and teacher networks, respectively. The matrix  $W_H \in \mathbb{R}^{d' \times d}$  serves as a learnable linear transformation that maps the hidden state of the student network into the same dimensional space as the hidden state of the teacher network.

Using the above distillation objective functions [i.e., (4) and (5)], the final distillation loss of the matching Transformer-layer, denoted as  $\mathcal{L}_{\text{Trans}}$ , in the network between the student and the teacher models can be expressed as follows:

$$\mathcal{L}_{\text{Trans}} = \mathcal{L}_{\text{MHA}} + \mathcal{L}_{\text{Hidn}}. \quad (6)$$

#### D. Fine-tuning using Data Transformation

During pretraining, the model is solely trained on unlabeled byte-level data-gram traffic, fostering its ability to acquire generalized representations of network traffic. The pretrained model can be efficiently leveraged for various downstream tasks. During fine-tuning, a minimal set of labeled traffic is utilized to adapt the pretrained model for specific downstream tasks. Prior to fine-tuning, traffic data must be converted into datagram byte tokens. The [CLS] from the pretrained model's output encapsulates the entire input traffic's representation.

Given the shared architecture between pretraining and fine-tuning, we refine all parameters by fine-tuning the pretrained model with known traffic data representations. For intrusion detection in IoT networks, where resources are scarce and real-time processing is crucial, a multiclass classifier leverages the [CLS] representation from the output layer to classify potential threats. To tackle these constraints, we propose an innovative fine-tuning approach that utilizes a sliding window as the input mechanism, enabling efficient and timely analysis of data streams. The sliding window approach of fine-tuning, depicted in Fig. 1, efficiently processes traffic data by maintaining a contextual window. This approach requires only a minimal number of packets ( $K = 5$ ) to form a window sequence, significantly reducing computational load and ensuring real-time performance. We utilize a continuous stream of  $K$  packets as input and extract window sequences with a stride of  $S = 1$ . Given that packets within a window sequence originate from the same known traffic dataset, each sequence is assigned a corresponding label for accurate analysis.

## IV. EXPERIMENTS

### A. Experimental Setup

1) **Datasets:** The experiment is conducted on three real-world IoT network traffic datasets: TON-IoT [20], Edge-IIoTset [21], and IoMT2024 [22] to validate the performance of the proposed method. The TON-IoT dataset integrates IoT and

**TABLE I**  
MULTICLASSIFICATION RESULTS OF TON-IOT

| Class            | student       |               |               | teacher       |               |               |
|------------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                  | Pr            | Rc            | F1            | Pr            | Rc            | F1            |
| DDoS             | 0.984         | 0.974         | 0.979         | 0.994         | 0.992         | 0.993         |
| Normal           | 0.998         | 0.990         | 0.994         | 1.000         | 0.950         | 0.998         |
| DoS              | 0.996         | 0.970         | 0.983         | 0.990         | 0.994         | 0.985         |
| Ransomware       | 0.876         | 0.872         | 0.874         | 0.898         | 0.996         | 0.908         |
| XSS              | 0.978         | 0.966         | 0.972         | 0.996         | 0.980         | 0.983         |
| Backdoor         | 0.855         | 0.894         | 0.874         | 0.897         | 0.918         | 0.908         |
| MITM             | 0.998         | 0.966         | 0.982         | 0.992         | 0.970         | 0.981         |
| Injection        | 0.931         | 0.912         | 0.921         | 0.958         | 0.920         | 0.954         |
| Password         | 0.899         | 0.928         | 0.913         | 0.950         | 0.970         | 0.949         |
| Scanning         | 0.942         | 0.974         | 0.958         | 0.957         | 0.948         | 0.969         |
| <b>Macro Avg</b> | <b>0.9457</b> | <b>0.9446</b> | <b>0.9450</b> | <b>0.9632</b> | <b>0.9638</b> | <b>0.9628</b> |
| <b>ACC</b>       | <b>0.9446</b> |               |               | <b>0.9626</b> |               |               |

Note: The boldface denotes the overall performance of the proposed method.

IIoT data, featuring ten labeled network data types like Normal, Scanning, denial of service/distributed denial of service (DoS/DDoS), Ransomware, etc. For our experiment, we chose a subset with the first pcap file from each attack type. Edge-IIoTset, a comprehensive IoT/IIoT cybersecurity dataset, simulates 14 attack types under five threat categories: DoS/DDoS, Information gathering, man-in-the-middle (MITM), Injection, and Malware attacks. IoMT2024, a medical device benchmark dataset, covers Wi-Fi, message queuing telemetry transport (MQTT), and Bluetooth protocols in healthcare. It categorizes attacks into five groups: DDoS, DoS, Reconnaissance, MQTT-related, Spoofing with 18 subtypes, and Bluetooth DoS. We use a subset (20%) of IoMT2024 for continue-pretraining.

**2) Data Preprocessing:** We replace the IP address with a randomly generated one, reset the port number to 0, and strip the Ethernet header of the packet to eliminate potential interference. As depicted in Fig. 1, we utilize raw traffic data as input for both pretraining and fine-tuning phases. During fine-tuning, we employ a sliding window algorithm to create sequences from the dataset, randomly selecting 5000 sequences per class. Each dataset is subsequently partitioned into training, validation, and test sets in an 8:1:1 ratio. During the fine-tuning phase, the validation set is utilized to assess the model's performance and facilitate the selection of optimal hyperparameters.

**3) Implementation Details:** In pretraining, we optimize with batch size 32, learning rate  $2e-5$ , 500k steps, and a 0.1 warm-up ratio. For knowledge distillation, we employ a compact student model comprising two layers, with a hidden dimension of 128, a feedforward size of 512, and two attention heads, with 9 million parameters. The teacher model BERT<sub>teacher</sub> ( $N=12$ ,  $d=768$ ,  $d_i=3072$ , and  $h=12$ ) contains 135 million parameters. The learning weight  $\lambda$  for each layer is set to 1 as each layer contributes critical linguistic knowledge to the student model. During the fine-tuning phase, we adopt a base learning rate of  $2 \times 10^{-5}$  with the AdamW optimizer for ten epochs, maintaining a batch size of 32 and a dropout rate of 0.5 for effective regularization. All experiments are conducted using NVIDIA Corporation device 2204 GPU, Pytorch 2.1.2, and universal encoder representations framework [23].

**4) Evaluation Measures:** To avoid the result deviation caused by data imbalance, we evaluate the model through four typical metrics: Accuracy (Acc), Precision (Pr), Recall (Rc),

**TABLE II**  
MULTICLASSIFICATION RESULTS OF EDGE-IIOTSET

| Class                | student       |               |               | teacher       |               |               |
|----------------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                      | Pr            | Rc            | F1            | Pr            | Rc            | F1            |
| Password             | 1.000         | 0.982         | 0.991         | 0.998         | 0.988         | 0.993         |
| Ransomware           | 0.988         | 0.976         | 0.982         | 0.984         | 0.990         | 0.987         |
| MITM                 | 0.827         | 0.943         | 0.881         | 0.928         | 0.951         | 0.939         |
| Port_Scanning        | 0.992         | 0.986         | 0.989         | 0.994         | 0.984         | 0.989         |
| SQL_injection        | 1.000         | 0.998         | 0.999         | 0.998         | 0.998         | 0.998         |
| DDoS_HTTP            | 0.996         | 1.000         | 0.998         | 0.998         | 1.000         | 0.999         |
| XSS                  | 0.965         | 0.980         | 0.972         | 0.978         | 0.986         | 0.982         |
| VulnerabilityScanner | 0.998         | 0.998         | 0.998         | 0.994         | 0.998         | 0.996         |
| Uploading            | 0.998         | 0.992         | 0.995         | 0.996         | 0.996         | 0.996         |
| DDoS_UDP             | 1.000         | 1.000         | 1.000         | 1.000         | 1.000         | 1.000         |
| DDoS_TCP             | 1.000         | 1.000         | 1.000         | 1.000         | 1.000         | 1.000         |
| Normal               | 0.996         | 1.000         | 0.998         | 0.998         | 1.000         | 0.999         |
| Fingerprinting       | 0.991         | 0.949         | 0.970         | 0.991         | 0.975         | 0.983         |
| DDoS_ICMP            | 1.000         | 1.000         | 1.000         | 1.000         | 1.000         | 1.000         |
| Backdoor             | 0.998         | 0.994         | 0.996         | 0.998         | 0.994         | 0.996         |
| <b>Macro Avg</b>     | <b>0.9833</b> | <b>0.9865</b> | <b>0.9846</b> | <b>0.9903</b> | <b>0.9907</b> | <b>0.9905</b> |
| <b>ACC</b>           | <b>0.9911</b> |               |               | <b>0.9938</b> |               |               |

Note: The boldface denotes the overall performance of the proposed method.

**TABLE III**  
MULTICLASSIFICATION RESULTS OF IoMT2024

| Class            | student       |               |               | teacher       |               |               |
|------------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                  | Pr            | Rc            | F1            | Pr            | Rc            | F1            |
| Recon VulScan    | 0.978         | 0.964         | 0.971         | 0.978         | 0.982         | 0.980         |
| Bluetooth Benign | 1.000         | 0.998         | 0.999         | 1.000         | 1.000         | 1.000         |
| DoS SYN          | 0.939         | 0.712         | 0.810         | 0.959         | 0.974         | 0.966         |
| MQTT DDoS Pub.   | 0.998         | 0.986         | 0.992         | 1.000         | 0.990         | 0.995         |
| DDoS SYN         | 0.768         | 0.956         | 0.852         | 0.972         | 0.956         | 0.964         |
| Recon Port Scan  | 1.000         | 0.988         | 0.994         | 0.996         | 0.992         | 0.994         |
| MQTT Malformed   | 0.982         | 0.988         | 0.985         | 0.984         | 0.992         | 0.988         |
| ARP Spoofing     | 0.931         | 0.976         | 0.953         | 0.992         | 0.972         | 0.982         |
| DDoS ICMP        | 0.988         | 0.998         | 0.993         | 1.000         | 0.998         | 0.999         |
| Recon Ping Sweep | 0.987         | 0.992         | 0.989         | 0.987         | 1.000         | 0.993         |
| DDoS TCP         | 0.499         | 0.706         | 0.584         | 0.921         | 0.750         | 0.827         |
| Benign           | 0.948         | 0.978         | 0.963         | 0.972         | 0.990         | 0.981         |
| MQTT DDoS Cnct   | 0.988         | 0.990         | 0.989         | 0.992         | 0.994         | 0.993         |
| MQTT DoS Pub.    | 0.998         | 0.998         | 0.998         | 1.000         | 1.000         | 1.000         |
| Recon OS Scan    | 0.996         | 0.964         | 0.980         | 0.994         | 0.984         | 0.989         |
| DDoS UDP         | 0.996         | 0.998         | 0.997         | 1.000         | 1.000         | 1.000         |
| DoS ICMP         | 0.998         | 0.992         | 0.995         | 0.998         | 0.996         | 0.997         |
| MQTT DoS Cnct    | 0.998         | 0.984         | 0.991         | 0.988         | 0.996         | 0.992         |
| DoS UDP          | 1.000         | 0.998         | 0.999         | 0.998         | 0.998         | 0.998         |
| DoS TCP          | 0.498         | 0.284         | 0.362         | 0.792         | 0.938         | 0.859         |
| Bluetooth DoS    | 0.998         | 1.000         | 0.999         | 1.000         | 1.000         | 1.000         |
| <b>Macro Avg</b> | <b>0.9280</b> | <b>0.9262</b> | <b>0.9236</b> | <b>0.9773</b> | <b>0.9763</b> | <b>0.9760</b> |
| <b>ACC</b>       | <b>0.9254</b> |               |               | <b>0.9760</b> |               |               |

Note: The boldface denotes the overall performance of the proposed method.

and F1-score ( $F1$ ) [24]. Meanwhile, we calculate their macro average to obtain an overall evaluation [25].

## B. Experimental Results and Comparative Analysis

Tables I–III present the multiclassification results of the comparison models, evaluated on the TON-IoT, Edge-IIoTset, and IoMT2024 datasets. The teacher model achieved an impressive overall accuracy ranging from 96.26% to 99.38%, demonstrating its superiority. Remarkably, the student model, with just 6% of the teacher's parameters, maintains comparable performance on TON-IoT and Edge-IIoTset datasets, exhibiting a minor 5.06% loss in accuracy on IoMT2024. This underscores the effectiveness of the knowledge distillation and the efficiency of the student model. As depicted in Fig. 3, the student model's confusion matrix for IoMT2024 indicates difficulties in distinguishing between “DDoS synchronize sequence numbers

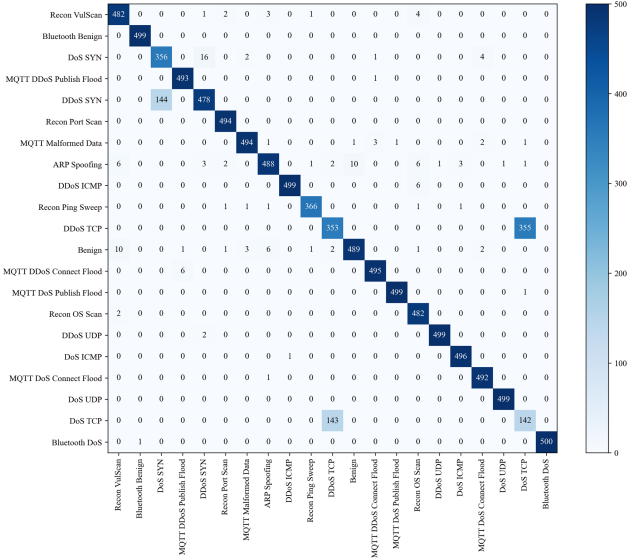


Fig. 3. Confusion matrix of student model on IoMT2024

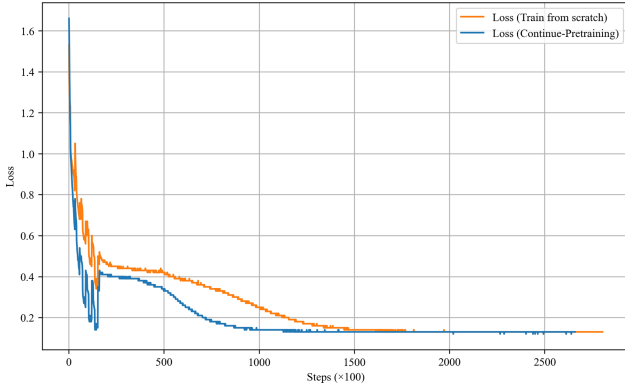


Fig. 4. Loss Curves: Pretraining with and without continue-pretraining in the IoMT2024 dataset

(SYN)” and “DoS SYN,” as well as “DDoS transmission control protocol (TCP)” and “DoS TCP” attacks. The key distinction between DDoS and DoS lies in the number of sources: DDoS involves multiple sources, whereas DoS originates from a single or limited number of sources, both aiming to deny network resources. Anonymizing IP addresses during preprocessing, while protecting privacy, may have inadvertently hindered the model’s ability to accurately differentiate these attack types on IoMT2024.

To validate the efficacy of our models, we compare them with several classical algorithms, specifically decision tree (DT), random forest (RF), support vector machine (SVM), k-nearest neighbor (KNN), and E-GraphSAGE [26], alongside some state-of-the-art methods, such as Generative AI [14], Security-BERT [15], FED-IDS [16], ET-BERT [18], Bayesian reinforcement learning-based explainable threat detection model (BRL-ETDM) [27], deep transfer learning (DTL) [28], DeepAK-IoT [29], and Deep-Packet [30]. As presented in Table IV, the proposed teacher model demonstrates superior performance, achieving the  $F1$  score of 0.9628, 0.9905, and 0.9760 across all three datasets. Notably, the student model closely matches the teacher model’s performance. The comparison results show that

our model achieves robust performance across datasets, surpassing classical methods that rely on manual feature engineering. DeepPacket and DeepAK-IoT perform well on specific datasets but show inconsistent results across diverse IoT traffic scenarios. Deep-Packet, designed for encrypted traffic classification, lacks specialized mechanisms to handle the heterogeneity of IoT network datasets and the nuanced features of diverse IoT protocols. DeepAK-IoT relies on manually engineered features for intrusion detection, which may overlook critical latent patterns or semantic correlations in data, leading to suboptimal results when dataset characteristics deviate from its predefined feature assumptions. In contrast, the proposed KD-BERT addresses these limitations by integrating adaptive feature learning and domain-specific complexity modeling, obtaining robust, dataset-agnostic performance across varied IoT traffic environments. Our approach can obtain an optimal tradeoff between performance and compactness.

### C. Ablation Study

To assess the model components, we perform ablation studies utilizing the Edge-IIoTset dataset. As shown in Table V, “HP” denotes removes header–payload pair tokenization strategy; “SW” means the sliding window technique; “KD” indicates the knowledge distillation module; “ $L_{MHA}$ ” and “ $L_{Hidn}$ ,” respectively, represent the two objective functions in knowledge distillation; “PT” represents pretraining task. The student model, featuring fewer parameters, outperforms both the teacher and nondistilled models, showcasing exceptional performance. These two distillation objective functions jointly contribute to distillation performance. The real-time performance of the student model is 0.034 ms per sequence on the GPU and 1.927 ms per sequence on the CPU. The removal of the sliding window notably impairs performance, highlighting its critical role in capturing sequence dependencies. The header-payload pairs tokenization strategy enhances feature learning, while the pretraining approach initializes model parameters to bolster generalization capabilities. To evaluate the efficacy of continue-pretraining, we compare three approaches on the IoMT2024 dataset: training a new BERT model from scratch, training a new BERT model from a mixed dataset containing the three datasets used, and continuing to train an Edge-IIoTset-pretrained BERT model. As demonstrated in Table VI and Fig. 4, continue-pretraining outperforms training from scratch by 4.79% and outperforms training on the mixed dataset by 4.95% in accuracy, while also achieving faster loss convergence. In scenarios with limited IoMT2024 data, training from scratch encounters challenges due to random initialization, whereas continue-pretraining effectively leverages existing knowledge to efficiently adapt to the new domain.

### D. Discussion

The proposed model avoids manual feature extraction and the problem of input dimension inconsistencies by directly leveraging raw network traffic as input. To address the dynamic nature of IoT network data caused by evolving scenarios and technological advancements, we implement the continue-pretraining strategy. Furthermore, knowledge distillation significantly reduces model parameters and computational complexity

TABLE IV  
COMPARISON RESULTS ON TON-IoT, EDGE-IIoTSET, AND IoMT2024 DATASETS

| Method             | #Paras | #FLOPs    | TON-IoT       |               |               |               | Edge-IIoTset  |               |               |               | IoMT2024      |               |               |               |
|--------------------|--------|-----------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                    |        |           | Acc           | Pr            | Rc            | F1            | Acc           | Pr            | Rc            | F1            | Acc           | Pr            | Rc            | F1            |
| DT                 | 17.6K  | 2.9M      | 0.9565        | 0.8487        | 0.7523        | 0.7523        | 0.8841        | 0.8666        | 0.8640        | 0.8652        | 0.9978        | 0.9451        | 0.9427        | 0.9436        |
| RF                 | 2.8M   | 654.7M    | 0.9604        | 0.8563        | 0.7714        | 0.7627        | 0.8831        | 0.8755        | 0.8630        | 0.8677        | <b>0.9981</b> | 0.9685        | 0.9367        | 0.9508        |
| SVM                | 50     | 1.2M      | 0.6631        | 0.4812        | 0.2008        | 0.2119        | 0.7148        | 0.6561        | 0.6478        | 0.6425        | —             | —             | —             | —             |
| KNN                | 85.6M  | 240751.8B | <b>0.9789</b> | 0.8979        | 0.8976        | 0.8975        | 0.7606        | 0.7453        | 0.7367        | 0.7394        | 0.9697        | 0.8857        | 0.8286        | 0.8512        |
| E-GraphSAGE [26]   | 89K    | 708M      | 0.8717        | 0.7648        | 0.7786        | 0.7419        | 0.9291        | 0.7322        | 0.8367        | 0.7739        | —             | —             | —             | —             |
| Generative AI [14] | —      | —         | —             | —             | —             | —             | 0.95          | 0.95          | 0.95          | 0.94          | —             | —             | —             | —             |
| SecurityBERT [15]  | 11M    | 1.6B      | —             | —             | —             | —             | 0.98          | 0.98          | 0.98          | 0.98          | —             | —             | —             | —             |
| FED-IDS [16]       | —      | —         | 0.9485        | 0.9317        | 0.9309        | 0.9313        | —             | —             | —             | —             | —             | —             | —             | —             |
| ET-BERT [18]       | 135M   | 22.4B     | 0.9538        | 0.9542        | 0.9538        | 0.9538        | 0.9350        | 0.9278        | 0.9264        | 0.9227        | 0.9213        | 0.9251        | 0.9221        | 0.9060        |
| BRL-ETDM [27]      | —      | —         | 0.9570        | 0.9186        | 0.9484        | 0.9333        | 0.9615        | 0.9595        | 0.9596        | 0.9596        | —             | —             | —             | —             |
| DTL [28]           | 0.33M  | 9.7B      | 0.87          | 0.88          | 0.86          | 0.86          | —             | —             | —             | —             | —             | —             | —             | —             |
| DeepAK-IoT [29]    | 0.5M   | 13.8B     | 0.9057        | 0.8959        | —             | 0.8887        | 0.9496        | 0.9540        | —             | 0.9472        | —             | —             | —             | —             |
| Deep-Packet [30]   | 3.5M   | 61.7M     | —             | 0.9557        | <b>0.9686</b> | 0.9614        | —             | 0.7322        | 0.9396        | 0.7860        | —             | 0.6897        | 0.8757        | 0.7018        |
| Ours (Teacher)     | 135M   | 22.4B     | 0.9626        | <b>0.9632</b> | 0.9638        | <b>0.9628</b> | <b>0.9938</b> | <b>0.9903</b> | <b>0.9907</b> | <b>0.9905</b> | 0.9760        | <b>0.9773</b> | <b>0.9763</b> | <b>0.9760</b> |
| Ours (Student)     | 9M     | 117.5M    | 0.9446        | 0.9457        | 0.9446        | 0.9450        | 0.9911        | 0.9833        | 0.9865        | 0.9846        | 0.9254        | 0.9280        | 0.9262        | 0.9236        |

Note: — means that the data are not available. The boldface denotes the best result.

TABLE V  
ABLATION STUDY OF THE PROPOSED METHODS

| System         | Inference Time (ms) |              | Edge-IIoTset  |               |               |               |
|----------------|---------------------|--------------|---------------|---------------|---------------|---------------|
|                | GPU                 | CPU*         | Acc           | Pr            | Rc            | F1            |
| Teacher        | 1.768               | 147.467      | <b>0.9880</b> | <b>0.9893</b> | <b>0.9880</b> | <b>0.9882</b> |
| w/o HP         | —                   | —            | 0.9833        | 0.9841        | 0.9833        | 0.9835        |
| w/o SW         | —                   | —            | 0.9377        | 0.9338        | 0.9293        | 0.9247        |
| Student        | <b>0.034</b>        | <b>1.927</b> | <b>0.9793</b> | <b>0.9806</b> | <b>0.9793</b> | <b>0.9795</b> |
| w/o HP         | —                   | —            | 0.9787        | 0.9805        | 0.9787        | 0.9789        |
| w/o SW         | —                   | —            | 0.9368        | 0.9281        | 0.9258        | 0.9226        |
| w/o KD         | —                   | —            | 0.9633        | 0.9652        | 0.9633        | 0.9635        |
| w/o $L_{MHA}$  | —                   | —            | 0.9780        | 0.9801        | 0.9780        | 0.9783        |
| w/o $L_{Hidn}$ | —                   | —            | 0.9773        | 0.9783        | 0.9773        | 0.9775        |
| w/o PT         | —                   | —            | 0.8967        | 0.8997        | 0.8967        | 0.8965        |

\*AMD Ryzen 7 5800H with Radeon Graphics. The boldface denotes the best result.

TABLE VI  
ABLATION STUDY OF CONTINUE-PRETRAINING MODULE

| System               | IoMT2024      |               |               |               |
|----------------------|---------------|---------------|---------------|---------------|
|                      | Acc           | Pr            | Rc            | F1            |
| Train from scratch   | 0.9281        | 0.9301        | 0.9290        | 0.9263        |
| Mixed dataset        | 0.9265        | 0.9286        | 0.9274        | 0.9270        |
| Continue-Pretraining | <b>0.9760</b> | <b>0.9773</b> | <b>0.9763</b> | <b>0.9760</b> |

The boldface denotes the best result.

while maintaining competitive performance on public datasets. However, this compression introduces performance tradeoffs, particularly in fine-grained attack detection. For instance, the model exhibits relatively low recognition accuracy for certain DDoS/DoS attacks in the IoMT2024 dataset.

## V. CONCLUSION

In this article, we propose KD-BERT, an end-to-end IoT intrusion detection method designed to operate efficiently in resource-constrained IoT environments. By leveraging raw traffic data and knowledge distillation, our approach eliminates the need for extensive feature engineering and makes it both lightweight and adaptable. We evaluate the effectiveness of the proposed method on three public datasets (TON-IoT, Edge-IIoTset, and IoMT2024). The experimental results demonstrate that our model achieves good detection performance across the three datasets and has strong generalization ability. Further analysis of the ablation study results shows that each modified

module contributed to model performance. The successful application of KD-BERT demonstrates the significant potential of knowledge distillation in IoT NIDS, while also highlighting its drawback: performance degradation. Future work will focus on exploring ways to minimize performance loss while improving the generalization capability of the student model.

## REFERENCES

- [1] L. D. Xu, W. He, and S. Li, "Internet of things in industries: A survey," *IEEE Trans. Ind. Inform.*, vol. 10, no. 4, pp. 2233–2243, Nov. 2014.
- [2] Y. Zhang, Y. Wu, and X. Huang, "Toward transferable adversarial attacks against autoencoder-based network intrusion detectors," *IEEE Trans. Ind. Inform.*, vol. 20, no. 12, pp. 13863–13872, Dec. 2024.
- [3] M. Lin, K. Yang, Z. Yu, Y. Shi, and C. L. P. Chen, "Hybrid ensemble broad learning system for network intrusion detection," *IEEE Trans. Ind. Inform.*, vol. 20, no. 4, pp. 5622–5633, Apr. 2024.
- [4] W. Zhou, Y. Jia, A. Peng, Y. Zhang, and P. Liu, "The effect of IoT new features on security and privacy: New threats, existing solutions, and challenges yet to be solved," *IEEE Internet things J.*, vol. 6, no. 2, pp. 1606–1616, Apr. 2018.
- [5] Z. Zhou, Y. Chen, and Y. Zhou, "Deep dynamic memory augmented attentional dictionary learning for image denoising," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 9, pp. 4784–4797, Sep. 2023.
- [6] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Inform.*, Beijing, China, 2017, pp. 43–48.
- [7] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for iot security based on learning techniques," *IEEE Commun. Surv. Tut.*, vol. 21, no. 3, pp. 2671–2701, 2019.
- [8] S. Roy, J. Li, B.-J. Choi, and Y. Bai, "A lightweight supervised intrusion detection mechanism for IoT networks," *Future Gener. Comput. Syst.*, vol. 127, pp. 276–285, 2022.
- [9] X. Jiao et al., "TinyBERT: Distilling bert for natural language understanding," in *Findings of the Association for Computational Linguistics*, 2020, pp. 4163–4174.
- [10] Z. Wang, Z. Li, D. He, and S. Chan, "A lightweight approach for network intrusion detection in industrial cyber-physical systems based on knowledge distillation and deep metric learning," *Expert Syst. Appl.*, vol. 206, 2022, Art. no. 117671.
- [11] Z. Wang, J. Li, S. Yang, X. Luo, D. Li, and S. Mahmoodi, "A lightweight IoT intrusion detection model based on improved BERT-of-Theseus," *Expert Syst. Appl.*, vol. 238, 2024, Art. no. 122045.
- [12] W. Xiao, J. Su, Y. Chen, and G. Cao, "Cross-scale-guided fusion transformer for disaster assessment using satellite imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4600512.
- [13] N. Liu, Q. Hu, H. Xu, X. Xu, and M. Chen, "Med-BERT: A pretraining framework for medical records named entity recognition," *IEEE Trans. Ind. Inform.*, vol. 18, no. 8, pp. 5600–5608, 2022.



- [14] M. A. Ferrag, M. Debbah, and M. Al-Hawawreh, "Generative AI for cyber threat-hunting in 6G-enabled IoT networks," in *Proc. IEEE Int. Symp. Cluster Cloud Internet Comput. Workshops*, 2023, pp. 16–25.
- [15] M. A. Ferrag et al., *Revolutionizing cyber threat detection with large language models: A privacy-preserving bert-based lightweight model for iot/iiot devices*. Piscataway, NJ, USA: IEEE Access, 2024.
- [16] M. Abdel-Basset, N. Moustafa, H. Hawash, I. Razzak, K. M. Sallam, and O. M. Elkomy, "Federated intrusion detection in blockchain-based smart transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2523–2537, 2021.
- [17] H. Y. He, Z. G. Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope: Ind.-Driven Digit. Transformation*, 2020, pp. 1–8.
- [18] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Berlin, Germany, 2022, pp. 633–642.
- [19] J. Lee et al., "BioBERT: A pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.
- [20] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON\_IoT datasets," *Sustain. Cities Soc.*, vol. 72, 2021, Art. no. 102994.
- [21] F. Li, J. Lin, Y. Wang, Y. Du, and H. Han, "Distributed hierarchical temporal graph learning for communication-efficient high-dimensional industrial iot modeling," *IEEE Internet Things J.*, vol. 11, no. 17, pp. 28578–28590, 2024.
- [22] S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. Ghorbani, "Ciciomt2024: A benchmark dataset for multi-protocol security assessment in iomt," *Internet Things*, vol. 28, 2024, Art. no. 101351.
- [23] Z. Zhao et al., "UER: An open-source toolkit for pre-training models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2019, pp. 241–246.
- [24] Z. Zhou, Y. Wu, and Y. Zhou, "Consistent arbitrary style transfer using consistency training and self-attention module," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 16845–16856, Nov. 2024.
- [25] Z. Zhou, Y. Chen, and Y. Zhou, "Simultaneously learning deep quaternion reconstruction and noise convolutional dictionary for color image denoising," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 2, pp. 1766–1779, Apr. 2025.
- [26] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-graphsage: A graph neural network based intrusion detection system for iot," in *Proc. IEEE Netw. Operations Manage. Symp.*, Berlin, Germany, 2022, pp. 1–9.
- [27] A. K. Dey, G. P. Gupta, and S. P. Sahu, "BRL-ETDM: Bayesian reinforcement learning-based explainable threat detection model for industry 5.0 network," *Cluster Comput.*, vol. 27, pp. 8243–8268, 2024.
- [28] S. T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, and R. Islam, "Dependable intrusion detection system for iot: A deep transfer learning based approach," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1006–1017, 2022.
- [29] W. Ding, M. Abdel-Basset, and R. Mohamed, "DeepAK-IoT: An effective deep learning model for cyberattack detection in IoT networks," *Inf. Sci.*, vol. 634, pp. 157–171, 2023.
- [30] A. Thakkar and R. Lohiya, "Attack classification of imbalanced intrusion data for IoT network using ensemble learning-based deep neural network," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11888–11895, Jul. 2023.



**Zhong Cao** received the M.S. degree in software and theory from the School of Computer Science and Educational Software, Guangzhou University, Guangzhou, China, in 2005, and the Ph.D. degree in detection technology and automation equipment from the School of Energy Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2015.

From December 2016 to December 2017, he was a Visiting Scholar with the Department of Civil and Environmental Engineering, University of North Carolina at Charlotte, Charlotte, NC, USA. He is currently an Associate Professor with the School of Electronics and Communication Engineering, Guangzhou University. His research interests include artificial intelligence, machine learning, industrial control system, and computer vision.



**Xiaohua Liu** was born in Guangdong, China in 2004. He is currently working toward the undergraduation degree in electronic information engineering with the School of Electronics and Communication Engineering, Guangzhou University, Guangzhou, China.

His research interests include artificial intelligence, machine learning, industrial control system, and computer vision.



**Zheng Zhou** received the B.S. degree in electronic science and technology from the University of Electronic Science and Technology of China, Chengdu, China, in 2017, and the Ph.D. degree in computer science from the University of Macau, Macau, China, in 2023.

He is currently a Lecturer with the School of Electronics and Communication Engineering, Guangzhou University, Guangzhou, China. His research interests include computer vision, image processing, feature extraction, and deep learning.



**Lei Ding** was born in Tianjin, China, in 1995. He is currently working toward the Ph.D. degree in cyberspace security with the School of Cyberspace Security, Guangzhou University, Guangzhou, China.

His research focuses on include industrial control system, information security, computational intelligence, and machine learning.



**Wenli Shang** was born in Heilongjiang, China, in 1974. He received the M.S. degree in mechanical design and theory from the School of Mechanical and Automation Engineering, Northeastern University, Boston, MA, USA, in 2002, and the Ph.D. degree in mechanical and electronic engineering from the Laboratory of Industrial Control Network and System, Shenyang Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2005.

From 2005 to 2019, he was an Assistant Researcher, Associate Researcher, and Researcher with the Shenyang Institute of Automation, Chinese Academy of Sciences. Since 2020, he has been a Professor with Guangzhou University, Guangzhou, China. His research interests include industrial control system information security, computational intelligence and machine learning, and edge computing.